Initiation à Linux



Contact :

pierre-antoine.jean

Linux is everywhere — we just don't see it



68%

des OS mobiles utilisent Android

(repose sur une version modifiée du noyau Linux)
\+ iOs repose sur Darwin, système Unix-like



63%

des sites web dont l'OS est connu utilisent Linux**

Infrastructure cloud

Archit. microservices

Systèmes

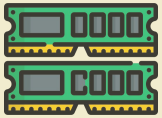
Cybersécurité

* https://gs.**statcounter**.com/os-market-share/mobile
** https://**w3techs**.com/technologies/breakdown/os-linux%20/web_server

Le système d'exploitation Linux

Un système d'exploitation (OS) est un **ensemble de programmes** qui coordonnent le fonctionnement des différents composants matériels et logiciels d'un système informatique

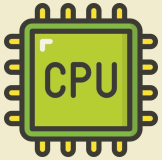
Le système d'exploitation gère :



L'allocation de la **mémoire** : partage de la mémoire entre les programmes



Les **périphériques** (écran, imprimante, disque dur, réseau): l'OS s'assure que les programmes puissent les utiliser de façon standard



Le **processeur** : utilisation partagée entre tous les programmes (planification des processus)



Les **utilisateurs** : gestion des droits d'accès aux fichiers et aux matériels

Le système d'exploitation Linux



Créé en 1991 par Linus Torvalds



Système open-source dérivé d'Unix



Existe de nombreuses distributions Linux

Une distribution Linux est une **suite de logiciels** assemblés **autour du noyau Linux** pour former un système d'exploitation pleinement opérationnel

Chaque distribution a sa propre philosophie et répond à des besoins spécifiques



Ubuntu : Commercialisé et maintenu par la société Canonical, cette distribution est **dérivée** d'une autre distribution nommée **Debian**. Elle propose tous les 6 mois une version stable (maintenue 9 mois) et tous les 2 ans une **version LTS** (maintenue plusieurs années). Parfait pour découvrir Linux.



ArchLinux : Distribution communautaire sans version, elle est en mise à jour permanente grâce à une communauté active de développeurs. Convient à un public davantage expérimenté.

Plan

1 Se repérer dans le système de fichiers

1.1 Le système de fichiers

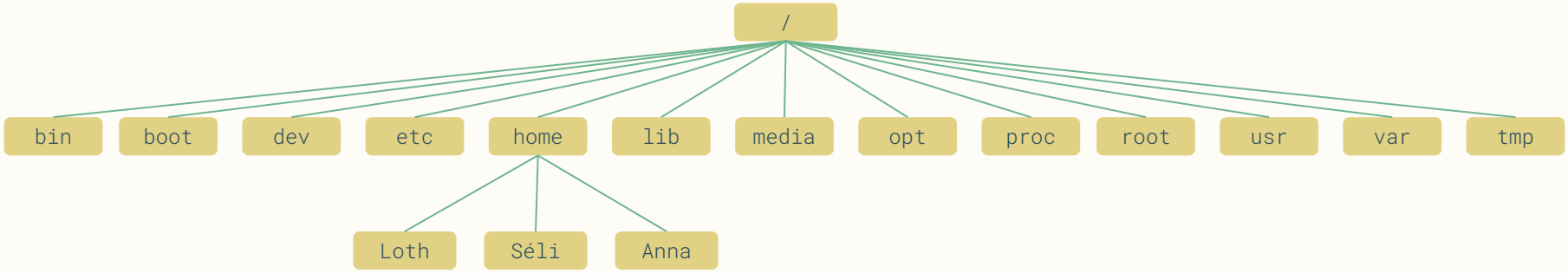
1.2 Les chemins d'accès

2 À la découverte du shell

3 Les commandes

Le système de fichiers

Le système de fichiers correspond à une arborescence que l'on parcourt de la racine (symbolisée par le caractère "/") vers les feuilles



/bin	Exécutables essentiels pour le système et utilisables par l'utilisateur
/boot	Fichiers permettant au système d'exploitation de démarrer
/dev	Points d'entrée des périphériques
/etc	Fichiers de configuration du système et du réseau
/home	Répertoires personnels des utilisateurs
/lib	Principales bibliothèques du système
/media	Points de montage des partitions temporaires (e.g. clés usb)
/opt	Packages des logiciels additionnels
/proc	Fichiers d'informations sur l'état du système et des processus en cours
/root	Répertoire de l'administrateur système
/usr	Données partagées par les utilisateurs du système
/log	Données fréquemment réécrites (e.g. logs)
/tmp	Fichiers temporaires

Les chemins d'accès

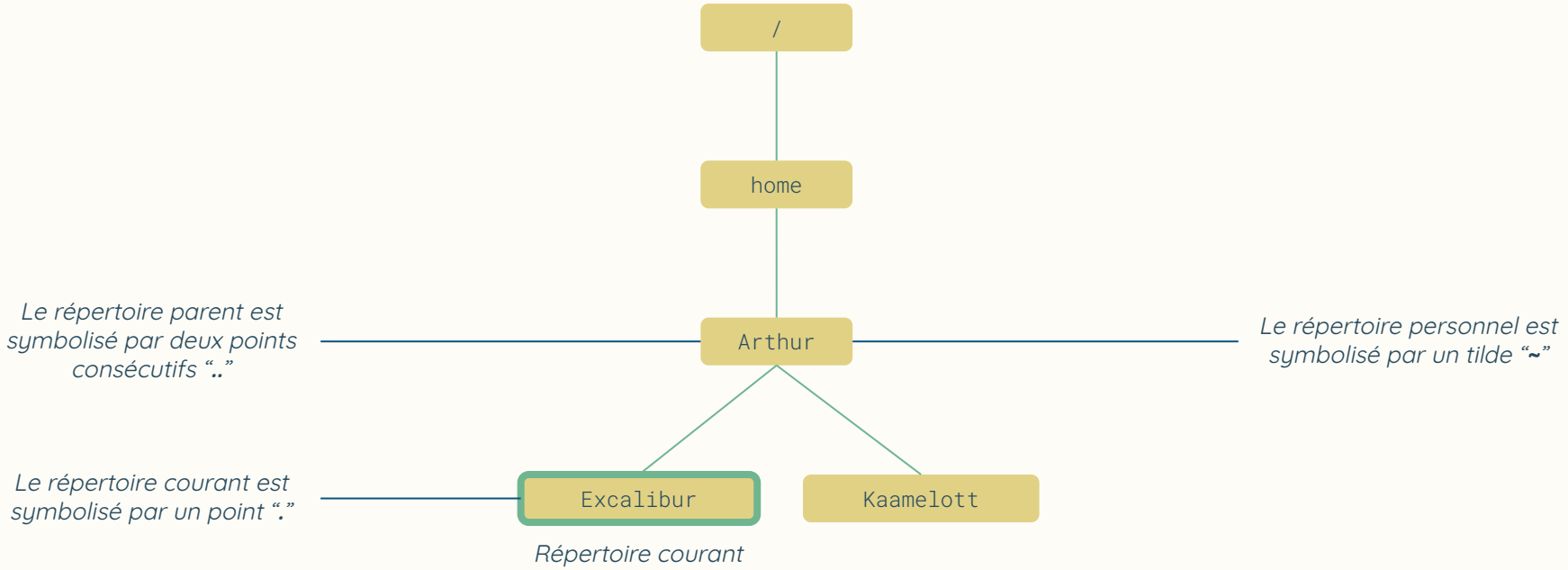
Un chemin d'accès est une suite de répertoires permettant de localiser précisément un fichier ou un répertoire dans l'arborescence du système de fichiers

Référence absolue correspond au chemin d'accès depuis la racine "**/**"

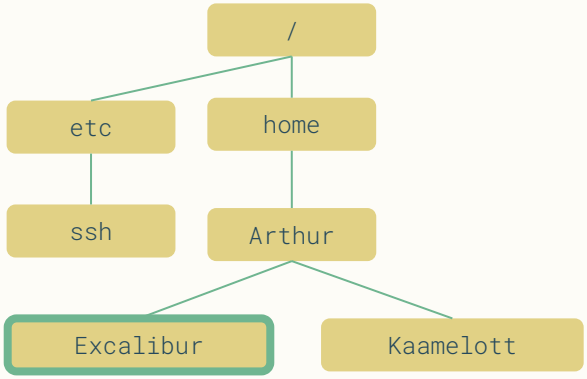
Example : /home/Arthur/Excalibur

Référence relative correspond au chemin d'accès depuis le répertoire de travail

Example : ../Kaamelott



Les chemins d'accès

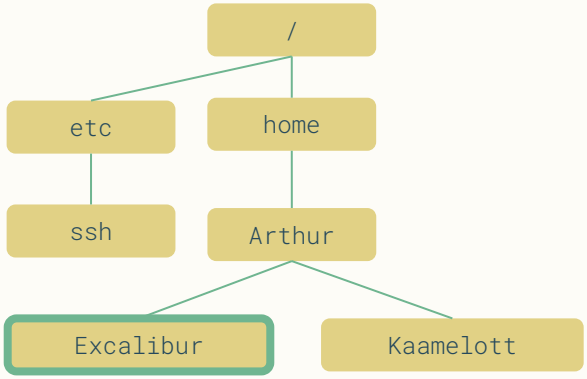


Répertoire courant

Le répertoire courant de l'utilisateur est Excalibur

1. Quelle est la référence absolue du répertoire ssh ?
2. À partir d'une référence relative, quel est le chemin d'accès au répertoire ssh?

Les chemins d'accès



Répertoire courant

Le répertoire courant de l'utilisateur est Excalibur

1. Quelle est la référence absolue du répertoire ssh ?

   /etc/ssh

2. À partir d'une référence relative, quel est le chemin d'accès au répertoire ssh?

   ../../../etc/ssh

Plan

1 Se repérer dans le système de fichiers

2 À la découverte du shell

2.1 Dialoguer avec le système

2.2 Les flux E/S et les redirections

3 Les commandes

Dialoguer avec le système

L'utilisateur dialogue avec le coeur du système (le noyau) par l'intermédiaire d'un interpréteur de commandes (ou **shell** tel que **Bash**) accessible via une interface textuelle appelée terminal



Un utilisateur saisit une commande

Exécution de la commande



Résultat de la commande **ls** pour lister le contenu d'un répertoire

Si l'utilisateur utilise l'interpréteur Bash, il peut retrouver l'**historique des commandes** qu'il a exécuté dans un fichier au sein de son répertoire personnel (**~/.bash_history**)

Les touches du clavier ↑ et ↓ permettent de faire défiler la liste des commandes précédemment tapées

ctrl + r permet d'effectuer une recherche sur l'historique

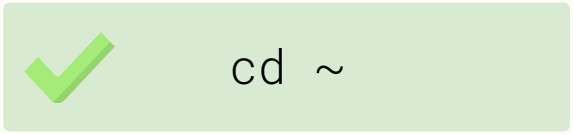
Utiliser la touche **tab** pour réaliser l'autocomplete de vos commandes ou chemins d'accès

Les flux E/S et les redirections

Une commande est un **programme**

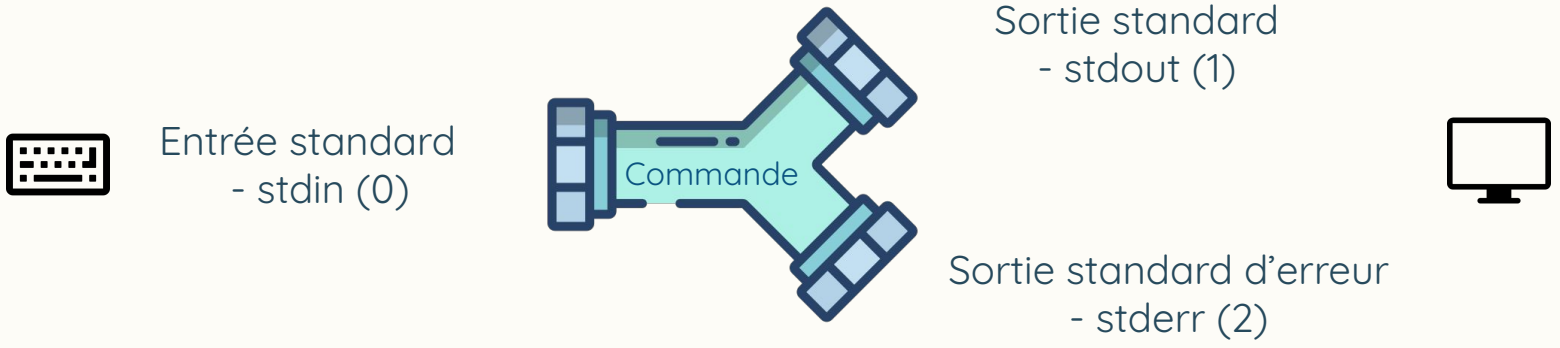
```
nom_commande [-liste_options] [liste_arguments]
```

Sensible à la **casse**





Une commande **crée un processus** et tous les processus possèdent **3 flux standards**: un d'entrée - stdin de valeur 0 - (typiquement le clavier) et deux de sortie (la sortie standard - stdout de valeur 1 - et l'erreur - stderr de valeur 2 - étant généralement l'affichage)



Les flux E/S et les redirections

> Redirection de la sortie standard

```
$ command > output.txt
```

Redirige la sortie standard de command dans le fichier output.txt (**écrase le contenu du fichier** ou **création du fichier** s'il n'existe pas)

2> Redirection de la sortie standard erreur

```
$ command 2> errors.txt
```

Redirige les messages d'erreurs de command dans le fichier errors.txt (écrase le contenu du fichier ou création du fichier s'il n'existe pas)

&> Redirection à la fois de la sortie standard et de la sortie standard erreur

```
$ command &> output.txt
```

Redirige la sortie standard de command ainsi que ses messages d'erreurs dans le fichier output.txt (écrase le contenu du fichier ou création du fichier s'il n'existe pas)

```
$ find / -name "*.conf" > output.txt 2> errors.txt
```

Redirection de stdout dans le fichier output.txt et de stderr dans le fichier errors.txt

Les flux E/S et les redirections

>> Redirection de la sortie standard avec concaténation (**ajoute à la suite** du fichier)

2>> Redirection de la sortie d'erreur standard avec concaténation (ajoute à la suite du fichier)

< Redirection de l'entrée standard

```
$ mysql database < sauvegarde.sql
```

Envoie le contenu du fichier SQL sur l'entrée standard de la commande **mysql**

| Redirection de la sortie standard d'une première commande vers l'entrée standard d'une seconde commande

```
$ ls -l | grep "Documents"
```

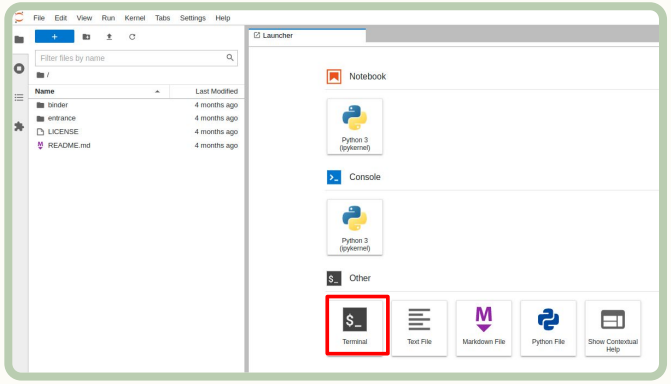
Le résultat de la commande **ls** est redirigé vers la commande **grep** qui permet de filtrer les lignes selon une chaîne de caractères ou une expression régulière

Serious Game, BashCrawler

Rendez-vous sur :

https://gitlab.com/slackermedia/bashcrawl

Dans la section “Try it online with mybinder”,
cliquer sur le lien pour accéder au JupyterLab



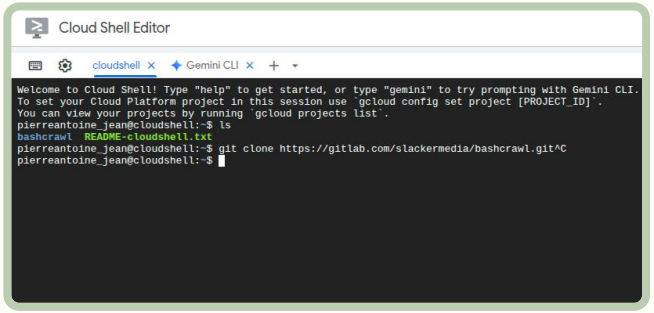
Ouvrir un nouveau terminal et
suivre les instructions !

Si la solution mybinder ne fonctionne pas rendez-vous sur :

https://shell.cloud.google.com

Puis entrer la commande suivante :

git clone https://gitlab.com/slackermedia/bashcrawl.git



Plan

1. Se repérer dans le système de fichiers
2. À la découverte du shell
3. Les commandes

RTFM

Afficher la documentation d'une commande (description, visualisation des options, etc.) avec la commande **man** (Manual)

```
$ man <nom_commande>
```

Parcourir l'arborescence de fichiers

Afficher le chemin d'accès absolu du répertoire courant (pwd = print working directory)

```
$ pwd
/home/Arthur
```

Afficher les informations des utilisateurs connectés au système (le nom de l'utilisateur, le terminal, la date de connexion)

```
$ who
Arthur   tty2   2026-09-16 9:15 (tty)
$ whoami
Arthur
```

Changer de répertoire

```
$ cd Documents
$ pwd
/home/Arthur/Documents
$ cd
$ pwd
/home/Arthur
```

cd sans argument redirige vers le répertoire personnel

```
$ ls
mon dossier
$ cd mon\ dossier
```

ou

```
$ cd "mon dossier"
```

Échappement de l'espace à l'aide d'un "\"

```
$ ls
-
$ cd ./-
```

ou

```
$ cd /home/Arthur/-
```

Expliciter un chemin d'accès pour éviter l'interprétation du "**-**"

Afficher les fichiers et répertoires

Afficher la liste des fichiers et répertoires

```
$ ls
fichier1.txt repertoire1 repertoire2
$ ls ~/Documents
image.png voyages
```

Afficher la liste **détaillée** des fichiers et répertoires

```
$ ls -l
-rw-rw-r-- 1 Arthur Arthur    0 sept. 16 11:34 fichier1.txt
drwxrwxr-x 3 Arthur Arthur 4096 sept. 16 11:24 repertoire1
```

- nom
- date de dernière modification
- taille
- groupe (accorde des droits à plusieurs utilisateurs en même temps)
- propriétaire
- nombre de liens (pour un répertoire chaque sous répertoire ajoute 1 lien physique via "..")
- permissions
- type (- = fichier, d = répertoire, l = lien symbolique)

Afficher la liste des fichiers et répertoires en **incluant les fichiers cachés** (débute avec un "**.**")

```
$ ls -a
.bash_rc fichier1.txt repertoire1 repertoire2
```

CRUD (create, read, update, delete)

Créer un répertoire

```
$ mkdir Projets
/home/Arthur
$ mkdir -p ./Projets/Tp1
```
l'option -p permet de créer les répertoires intermédiaires s'ils n'existent pas

```
$ mktemp -d
```
créer un répertoire temporaire au sein de /tmp

Créer un fichier vide

```
$ touch tp1.txt
```

```
$ mktemp
```
créer un fichier temporaire

Afficher le contenu d'un fichier avec la commande cat (concatenate)

```
$ cat fichier.txt
Les données du fichier
```

Affiche les données de fichier.txt

```
$ cat file1.txt file2.txt
Les données de file1
Les données de file2
$ cat file1.txt file2.txt > file3.txt
```

Affiche le contenu de plusieurs fichiers à la suite dans le terminal

Redirige le résultat dans un nouveau fichier

```
$ cat > nouveau_fichier.txt
```

Permet d'écrire directement dans un fichier, puis d'enregistrer avec **ctrl + d**

CRUD (create, read, update, delete)

Afficher le contenu ligne par ligne ou page par page d'un fichier

```
$ more tp1.txt
```

Afficher les *X* premières lignes d'un fichier

```
$ head -10 tp1.txt
```

Afficher les *X* dernières lignes d'un fichier

```
$ tail -10 tp1.txt
```

Afficher la nature des données contenues dans un fichier

```
$ file tp1.txt
```
(ex: ASCII text, PDF document, Microsoft Word 2007+, data, etc.)

Afficher sur la sortie standard des messages passés en paramètre après interprétation par le terminal (affichage des valeurs des variables d'environnement)

```
$ echo -e "Bonjour:\n$USER"
```
l'option -e permet d'interpréter les caractères échappés (\n, \t, etc.)

Modifier un fichier avec un éditeur

```
$ nano tp1.txt
```

```
$ (x)emacs tp1.txt
```

```
$ vi(m) tp1.txt
```

```
$ pico tp1.txt
```

CRUD (create, read, update, delete)

Copier un fichier

```
$ cp tp1.txt ../
```
copie le fichier tp1.txt dans le répertoire parent

Copier un répertoire

```
$ cp -R Documents /tmp/
```
copie l'arborescence du répertoire Documents dans le répertoire /tmp/

Déplacer un fichier ou un répertoire

```
$ mv tp1.txt ../
```

Renommer un fichier ou un répertoire

```
$ mv tp1.txt tp_1.txt
```

Supprimer un fichier

```
$ rm tp1.txt
$ rm ~/Documents/*.png
```
supprime tous les fichiers avec l'extension .png du répertoire ~/Documents

Supprimer un répertoire

```
$ rmdir ~/Documents/Projets
```
supprime un répertoire **vide**

```
$ rm -r ~/Documents/Projets
```
supprime de manière récursive un répertoire et son contenu

Connexion à un serveur distant

La commande ssh (Secure SHell) est un programme et un protocole de communication sécurisée, elle permet de se connecter à un serveur/système distant afin d'obtenir un terminal (nécessite un client ssh en local et un serveur ssh sur le serveur distant)

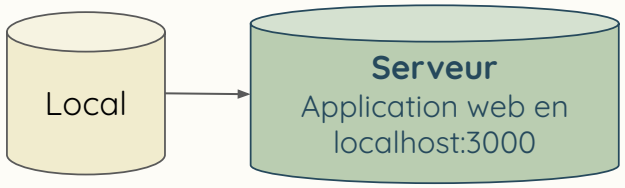
```
$ ssh <nom_utilisateur>@<adresse_ip>
```

Par défaut, SSH utilise le **port TCP 22** mais l'**option -p** permet de préciser un autre port de connexion

```
$ ssh -p 2220 <nom_utilisateur>@<adresse_ip>
```

Avec l'option -L, vous pouvez effectuer une redirection de port

ssh -L [port_local]:[hôte_distant]:[port_distant] utilisateur@serveur




```
$ ssh <utilisateur>@<ip>
```

Impossible d'accéder à l'application web depuis le local

```
$ ssh -L 3000:localhost:3000 <utilisateur>@<ip>
```

L'application web est accessible depuis le local via http://localhost:3000

Connexion à un serveur distant

Configurer des connexions ssh personnalisées via le fichier ~/.ssh/config

```
Host mon_serveur
        HostName 192.168.1.5
        IdentityFile ~/.ssh/ssh_key
        User docker
```

```
$ ssh mon_serveur
```

```
$ ssh -F ./mon_repertoire/config <nom>@<ip>
```
utilise un autre fichier de configuration ssh

Utiliser une clé privée pour se connecter

```
$ ssh -i sshkey.private <nom_utilisateur>@<adresse_ip>
```

Exécuter une commande sur un serveur distant

```
$ ssh <nom_utilisateur>@<adresse_ip> ls /
```

La commande **scp** (secure copy protocol) permet de réaliser des transferts sécurisés via une connexion ssh

```
$ scp -r -P 2220 <utilisateur>@<ip>:<chemin_serveur> <chemin_destination>
```

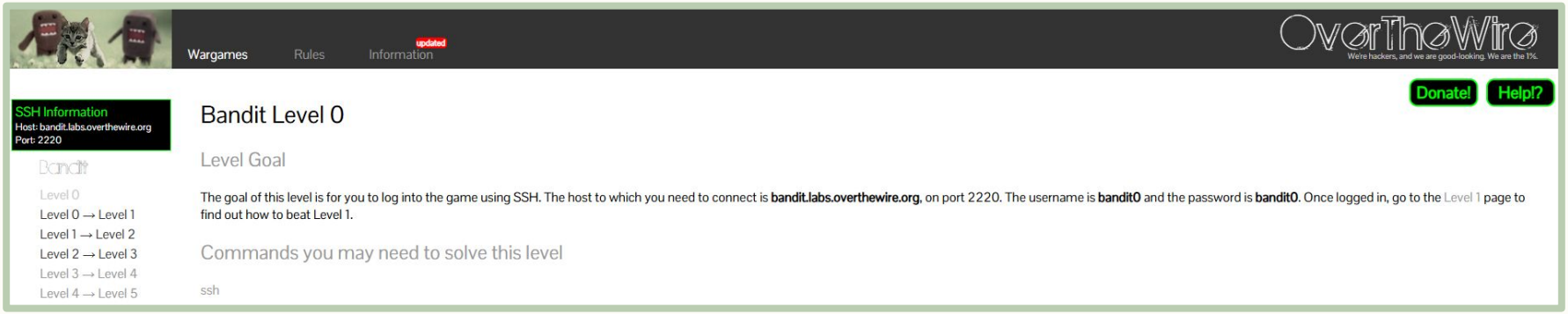
L'option **-r** permet de transférer des répertoires
L'option **-P** permet de modifier le numéro de port pour la connexion ssh

Serious Game, OverTheWire

Rendez-vous sur **https://overthewire.org/wargames/bandit/bandit0.html**

Suivre les instructions et accéder jusqu'au niveau "**Bandit Level 4 → Level 5**"

NB : Pour changer de niveau, vous devez vous déconnecter (commande **exit** ou **ctrl+d**) puis vous reconnecter avec le nouveau nom d'utilisateur (bandit0, bandit1, etc.)



Activer le client SSH pour Windows (si ce n'est pas déjà le cas) :

- Allez dans « Paramètres > Applications > « Fonctionnalités facultatives » et vérifier si le client OpenSSH est installé. Si ce n'est pas le cas, vous pouvez l'installer en cliquant sur « Ajouter une fonctionnalité facultative »
- Ouvrez **cmd** (invite de commandes) de Windows et tapez la commande ssh

Rechercher au sein de l'arborescence

La commande **find** permet d'effectuer une recherche sur l'arborescence

find [où rechercher] [quoi filtrer] [quoi faire]

```
$ find /home/Arthur -name "*.log" -exec rm {} \;
```

- `/home/Arthur` : Le répertoire de départ pour effectuer la recherche
- `-name "*.log"` : Filtre uniquement les fichiers dont le nom termine par .log
- `-exec rm {} \;` : Exécute la commande rm sur les fichiers filtrés

Quoi filtrer :

-name	<nom du fichier>	
-type	f / d	f = fichier, d = répertoire
-size	+100M -1k 10c	Taille supérieur à 100 Mo Taille inférieur à 1 ko Taille égale à 10 octets
-mtime	-7 +30	Modifié il y a moins de 7 jours Modifié il y a plus de 30 jours

-user	<nom>	Appartient à l'utilisateur <nom>
-group	<groupe>	Appartient au groupe <groupe>
-executable		Filtre les fichiers pouvant être exécutés par l'utilisateur
-not (ou !)		Applique un opérateur de négation au filtre suivant

Rechercher au sein de l'arborescence

L'option -exec permet d'exécuter une commande sur les résultats

```
$ find /home/Arthur -name "*.log" -exec rm {} \;
```

rm → commande exécutée

{} → emplacement des fichiers trouvés

\; fin de la commande (**;** pour indiquer à find que la commande est terminée et **** pour échapper le **;** car c'est un caractère spécial du terminal indiquant l'exécution de plusieurs commandes à la suite (exemple : echo "bonjour" **;** echo "au revoir")

```
$ find /home/Arthur -name "*.log" -exec cat {} +
```

+ → indique également la fin de la commande mais demande à find de regrouper les résultats

Examples :

\;
cat ./a.log
cat ./b.log
cat ./c.log

+
cat ./a.log ./b.log ./c.log

Manipuler le contenu d'un fichier

Compter dans un fichier avec la commande **wc** (word count)

```
$ wc -l tp1.txt
```
compte le nombre de lignes

```
$ wc -w tp1.txt
```
compte le nombre de mots

```
$ wc -m tp1.txt
```
compte le nombre de caractères

```
$ wc -c tp1.txt
```
compte le nombre d'octets

Trier les lignes d'un fichier

```
$ sort tp1.txt
```
trie alphanumérique croissant

```
$ sort -r tp1.txt
```
trie alphanumérique décroissant

Personnages.txt

```
Yvain|Gauvain
Perceval|Karadoc
Merlin|Elias
```

```
$ sort Personnages.txt
```

```
Merlin|Elias
Perceval|Karadoc
Yvain|Gauvain
```

Personnages.txt

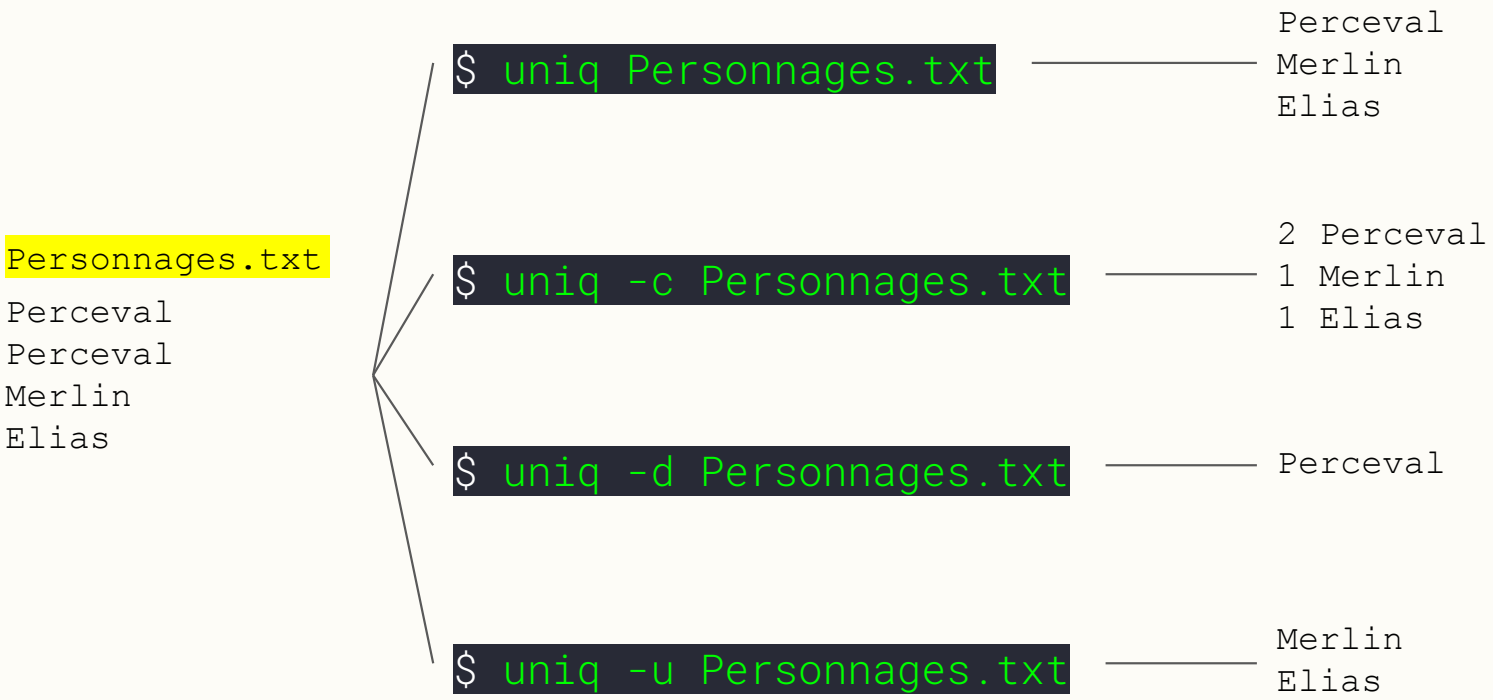
```
Yvain|Gauvain
Perceval|Karadoc
Merlin|Elias
```

```
$ sort -t'|' +1.1 Personnages.txt
```

```
Perceval|Karadoc
Yvain|Gauvain
Merlin|Elias
```

Manipuler le contenu d'un fichier

Supprimer les **doublons qui se suivent** dans un fichier texte avec la commande **uniq**



Manipuler le contenu d'un fichier

Rechercher une chaîne de caractères dans un fichier

```
$ grep "Couhillère" tp1.txt
```
affiche les lignes contenant le mot "Couhillère"

```
$ grep -i "Couhillère" tp1.txt
```
l'option **-i** ignore la casse

```
$ grep -v "couhillère" tp1.txt
```
affiche les lignes **ne contenant pas** la chaîne

```
$ grep -c "couhillère" tp1.txt
3
```
compte le nombre de lignes contenant la chaîne

```
$ grep -n "couhillère" tp1.txt
1:couhillère
2:couhillère
3:couhillère
```
chaque ligne contenant la chaîne est numérotée

```
$ grep -P "couhil+ère" tp1.txt
```
système étendu d'expression régulière (perl)

```
$ grep -o "cou" tp1.txt
```
affiche uniquement le motif correspondant contenu au sein d'une ligne

```
$ grep -a "couhillère" tp1.txt
```
traite un fichier binaire comme s'il s'agissait d'un fichier texte

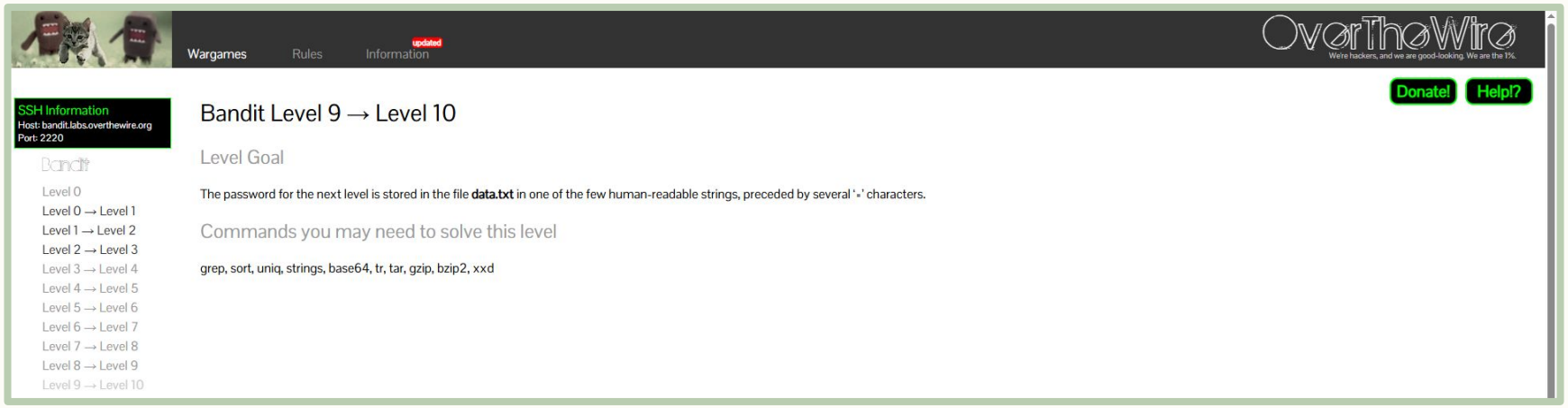
Extraire d'un fichier (généralement binaire) les suites de caractères imprimables (lisibles)

```
$ strings tp1.bin
k8c=
yo=-
```

Serious Game, OverTheWire

Rendez-vous sur https://overthewire.org/wargames/bandit/bandit0.html

Suivre les instructions et accéder jusqu'au niveau "**Bandit Level 9 → Level 10**"



Aide :
xxd -r <fichier> permet de transformer un hexdump (représentation textuelle des données brutes d'un fichier) en binaire

Manipuler le contenu d'un fichier

Afficher des zones spécifiques d'un fichier avec la commande **cut**

```
$ cut -c1-5 tp1.txt
```
affiche les colonnes de 1 à 5

```
$ cut -d';' -f2 tp1.txt
```
affiche la colonne 2 en utilisant le séparateur "**;**"

Personnages.txt

```
Perceval;1
Merlin;2
Elias;3
```

```
$ cut -d';' -f2 Personnages.txt
```

```
1
2
3
```

Convertir une chaîne de caractères avec la commande **tr** (translate or transliterate)

```
$ echo "abcd" | tr 'a' 'b'
bbcd
```
convertir tous les caractères "a" en caractère "b"

```
$ echo "abcd" | tr -c 'a' 'b'
abbbb
```
convertir tous les caractères qui ne sont pas "a" en caractère "b" (l'exemple inclut le saut de ligne \n)

```
$ echo "abcd" | tr -d 'a'
bcd
```
effacer un caractère spécifié

```
$ echo "les pattes de canaaaaaard" | tr -s 'a'
les pattes de canard
```
réduire à une unité un caractère spécifié qui se répète plusieurs fois de suite

```
$ tr 'a' 'b' < data.txt
```
utilise le contenu du fichier data.txt en entrée de la commande

```
$ tr '[a-z]' '[A-Z]' < data.txt
```
remplace les minuscules par des majuscules

Les expressions régulières

Début et fin de chaîne	^Bonjour	Débute par Bonjour
	bye$	Se termine par bye
Opérateur OU	Bonjour|bye	Recherche Bonjour ou bye
Ensemble de caractères	m[oai]ts	mots|mats|mits
	m[^oai]ts	Tout caractère différent de o, a ou i.
	[a-z]	[abcdefghijklmonpqrstuvwxyz]
	[A-Z]	[ABCDEFGHIJKLMNOPQRSTUVWXYZ]
	[0-9]	[0123456789]
	[a-z0-9]	[abcdefghijklmonpqrstuvwxyz0123456789]
	.	N'importe quel caractère
	\w	[a-zA-Z0-9_]
	\d	[0-9]
Les quantificateurs	{min, max}	Le caractère précédent est répété entre min et max fois
	{min,}	Le caractère précédent est au moins répété min fois
	{,max}	Le caractère précédent est au plus répété max fois
	{nombre}	Le caractère précédent est répété exactement nombre fois
	*	{0,}
	+	{1,}
	?	{,1}
Echappement	johndoe\.[a-z]{2,3}	Echappe le “.”

Les expressions régulières

Rendez-vous sur **https://regex101.com/** et copier/coller le texte présent dans le fichier regex.txt du répertoire GitHub (à partir de l'affichage Raw).

Répondez aux questions suivantes :

- Surligner les dialogues du Père Blaise.
- Surligner les dialogues du Père Blaise et de Léodagan.
- Surligner toutes les phrases qui ne commencent pas par un "P".
- Surligner les dialogues du Père Blaise qui se terminent par un "?".
- Surligner les chiffres au sein des dialogues.
- Surligner les numéros de téléphone.
- Surligner les adresses mail.

L'archivage et la compression

L'**archivage** consiste à rassembler plusieurs fichiers en un seul

```
$ tar -cf documents.tar ~/Documents/
```
archive le répertoire ~/Documents et ses sous-répertoire

```
$ tar -xf documents.tar
```
désarchive documents.tar

```
$ tar -zcf documents.tgz ~/Documents/
```
Archive et compresse (avec gzip)

La **compression** d'un fichier consiste à **diminuer l'espace disque** qu'il occupe (le principe de base est d'éviter les redondances en transformant une suite de bits A en une suite de bits B plus courte pouvant restituer les mêmes informations)

Outils de compression de fichier unique (algorithmes de compression différents)

```
$ gzip tp1.txt
```
compresse le fichier tp1.txt en tp1.txt.gz

```
$ gunzip tp1.txt.gz
```
décompresse le fichier tp1.txt.gz en tp1.txt

```
$ gunzip -c tp1.txt.gz
```
décompresse le fichier tp1.txt.gz et envoie le contenu sur **stdout**

```
$ bzip2 tp1.txt
```
compresse le fichier tp1.txt en tp1.txt.bz2

```
$ bunzip2 tp1.txt.bz2
```
décompresse le fichier tp1.txt.bz2 en tp1.txt

```
$ bunzip2 -c tp1.txt.bz2
```
décompresse le fichier tp1.txt.bz2 et envoie le contenu sur **stdout**

zip est un outil d'**archivage** et de **compression**

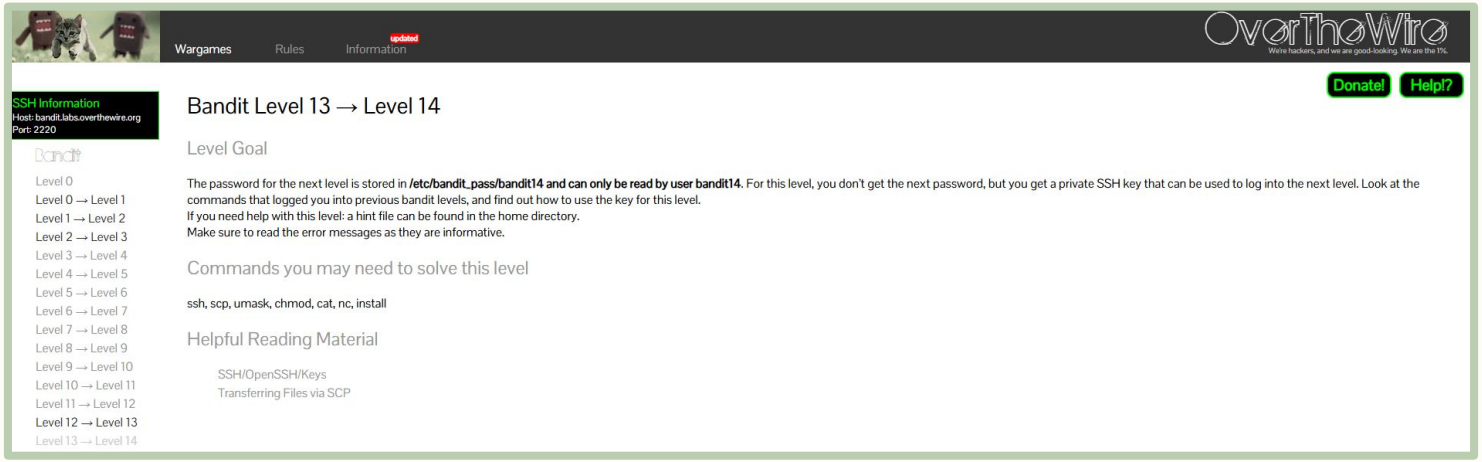
```
$ zip documents.zip ~/Documents/
```
archive et compresse ~/Documents

```
$ unzip documents.zip
```
désarchive et décompresse

Serious Game, OverTheWire

Rendez-vous sur https://overthewire.org/wargames/bandit/bandit0.html

Suivre les instructions et accéder jusqu'au niveau "**Bandit Level 13 → Level 14**"



Le système de droits

L'utilisateur Perceval réalise 2 commandes sur un répertoire dont il n'a pas les droits

```
Perceval@machine $ cd /home/Arthur
cd: /home/Arthur/: Permission denied
Perceval@machine $ ls /home/Arthur
ls: /home/Arthur/: Permission denied
Perceval@machine $ ls -l /home
drwxr-xr-x 29 Perceval groupe1 4096 juil. 25 09:00 Perceval/
drwx------ 29 Arthur groupe2 4096 mai. 21 18:30 Arthur/
```

nécessite le droit d'exécution (x)

nécessite le droit de lecture (r)

Il existe 3 types de droit applicables à 3 classes d'utilisateurs

- **r** : droit de lecture dans un fichier
- **w** : droit d'écriture dans un fichier
- **x** : droit d'exécution
- **-** : absence d'un droit

Répétés 3 fois pour indiquer le(s) personne(s) impliquée(s)

→ Le propriétaire (Perceval)

```
drwxr-xr-x
```

→ Les utilisateurs appartenant au même groupe que le propriétaire (groupe1)

```
drwxr-xr-x
```

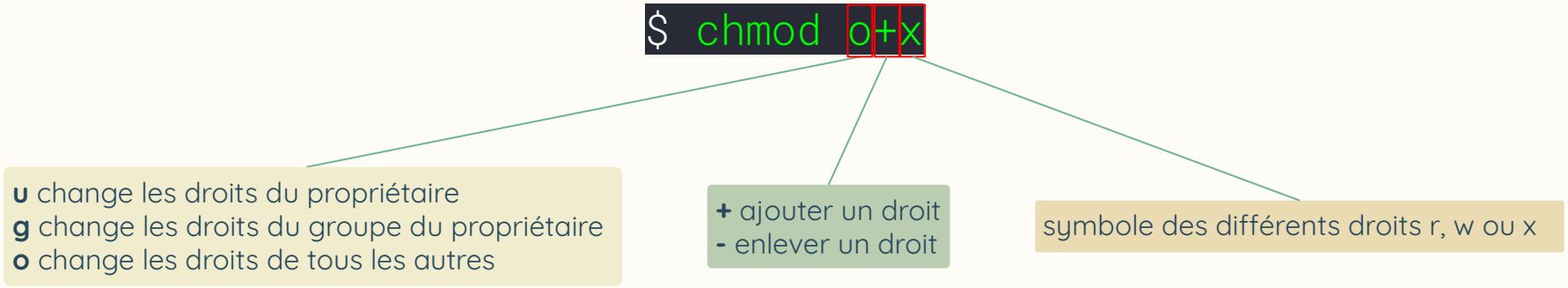
→ Le reste des utilisateurs

```
drwxr-xr-x
```

Le système de droits

chmod (change mode) permet de changer les droits d'un fichier/répertoire, uniquement le propriétaire et le root peuvent changer les droits d'un fichier/répertoire

```
$ chmod o+x /home/Arthur
```
cette commande exécutée par Arthur accorde le droit d'exécution à tous les utilisateurs sur son répertoire personnel



chown sert à modifier le propriétaire et/ou le groupe d'un fichier ou d'un répertoire

```
$ sudo chown alice fichier.txt
```
change uniquement le propriétaire
```
$ sudo chown :equipe fichier.txt
```
change uniquement le groupe
```
$ sudo chown alice:equipe fichier.txt
```
change le propriétaire et le groupe en même temps

sudo (superuser do) permet à un utilisateur autorisé d'exécuter une commande avec les privilèges d'administrateur (root)

Automatiser des tâches

cron permet d'exécuter automatiquement des commandes à des horaires définis

`$ crontab -l` liste la liste des tâches de l'utilisateur actuel
`$ crontab -e` édite la liste des tâches de l'utilisateur actuel
`59 23 * * * echo "cron" >> ~/Documents/tp1.txt &> /dev/null` tous les jours à 23h59 ajouter le mot "cron" à la suite du fichier tp1.txt sans notification

Pour réaliser des tâches cron sur le système (par exemple avec les droits de l'utilisateur root), les tâches peuvent être définies dans des fichiers au sein du répertoire **/etc/cron.d**

```
/etc/cron.d/
├── backup
├── logrotate
├── monitoring
└── my-task
```

Format d'une tâche système

```
# ┌──────────── minute
# │ ┌────────── heure
# │ │ ┌──────── jour du mois
# │ │ │ ┌────── mois
# │ │ │ │ ┌──── jour de la semaine
# │ │ │ │ │
# * * * * * utilisateur commande
```

`$ 0 2 * * * root /usr/local/bin/backup.sh` exécute backup.sh tous les jours à 02:00, avec les droits de root

Gestion des processus

ps permet d'afficher des informations sur une sélection d'instances de programmes en cours d'exécution (appelées processus), chacune identifiée par un PID (process ID)

```
$ ps -e
```
affiche tous les processus avec leur PID

```
$ ps -u
```
affiche la liste détaillée des processus dont l'utilisateur est propriétaire

```
Arthur      3082  0.0  0.0  14144  5504 pts/0    Ss   14:55   0:00 /bin/bash
Arthur      3321  0.0  0.0  15640  3568 pts/0    R+   15:00   0:00 ps -u
```

top permet de lister en **temps réel** les processus et les ressources utilisées

kill sert à envoyer un signal à un processus pour demander son arrêt

```
$ kill <PID>
$ kill -9 <PID>
```
arrêt forcé du processus



Une commande en cours d'exécution peut être arrêtée avec la manipulation **ctrl-c** (en mode interactif)

Lancer une commande en tâche de fond avec le symbole "**&**" (pour continuer à utiliser le shell) mais s'arrête lors de la fermeture du terminal/session (ex. ssh)

nohup exécute un programme et le rend résistant à la fermeture du terminal/session

```
$ nohup python3 serveur.py &> serveur.log &
```

Le gestionnaire de paquets apt

Un gestionnaire de paquets est un système qui permet d'installer des logiciels, de les maintenir à jour et de les désinstaller

apt (Advanced Packaging Tool) est le gestionnaire de paquets de la distribution Debian (et ses dérivés dont Ubuntu)

`$ sudo apt update` recherche les mises à jour des paquets installés

`$ sudo apt search <mot_cle>` recherche les paquets disponibles en fonction d'une expression

`$ sudo apt install tree` installe un paquet à partir de son nom

`$ sudo apt remove tree` supprime un paquet installé sur l'ordinateur

Outils réseau et cryptographique

Commandes	Fonction principale	Exemple d'utilisation
nc (Netcat)	Établir des connexions TCP/UDP et échanger des données	Écouter sur un port, tester une connexion
`$ nc -l 1234`	Écoute sur le port 1234	
nmap	Découvrir et analyser des hôtes/services réseau	Scanner les ports ouverts d'une machine
`$ nmap -p 22,80,443 192.168.1.10` `$ nmap -p 10000-10100`	Recherche les ports ouverts	
openssl	Cryptographie et protocoles TLS/SSL	Tester TLS, inspecter un certificat, chiffrer/déchiffrer
`$ openssl s_client -connect example.com:443 -quiet`		Établit une connexion TLS et affiche les informations du certificat

Outils réseau et cryptographique

nc permet de connecter l'entrée/sortie d'un terminal à une connexion réseau (TCP ou UDP) et peut donc servir à écouter ou à se connecter à un port

```
$ nc -l 1234
```
dans l'attente d'une connexion sur le port 1234

```
$ nc localhost 1234
```
connexion au port 1234 en localhost



Une fois la connexion établie, ce qu'on écrit d'un côté arrive de l'autre côté (la transmission des données n'est pas cryptée)

Lorsqu'on tape "bonjour" dans le terminal du **nc -l**, Netcat récupère bonjour sur stdin et l'envoie sur le socket

```
$ echo "Bonjour" | nc -l 1234
```
envoie le message "Bonjour" lors de la connexion au port 1234

La commande permet également de tester si un port est ouvert (mais nmap est plus complet pour scanner les ports d'une adresse)

```
$ nc -z -v www.google.com 80
Connection to www.google.com (2001:4860:4828:7700::) 80 port [tcp/http] succeeded!
```

Outils réseau et cryptographique

nmap permet d'observer ce qui est exposé sur un réseau

Identifier les machines accessibles sur un réseau

```
$ nmap -sn 192.168.1.0/24
Nmap scan report for 192.168.1.10
Host is up
Nmap scan report for 192.168.1.20
Host is up
Nmap scan report for 192.168.1.35
Host is up
```

Identifier les ports ouverts

`$ nmap 192.168.1.10` réalise un scan classique et teste un ensemble de ports TCP courants
`$ nmap -p 22,80,443 192.168.1.10` cible des ports spécifiques
`$ nmap -p 30000-30100 192.168.1.10` cible un ensemble de ports

Identifier les services et leur version

```
$ nmap -sV 192.168.1.10
PORT    STATE SERVICE VERSION
22/tcp  open  ssh     OpenSSH 8.9p1 Ubuntu
80/tcp  open  http    Apache httpd 2.4.52
443/tcp open  https   nginx 1.18.0
```

Outils réseau et cryptographique

openssl est une boîte à outils pour la cryptographie et les communications TLS/SSL

Cryptographie

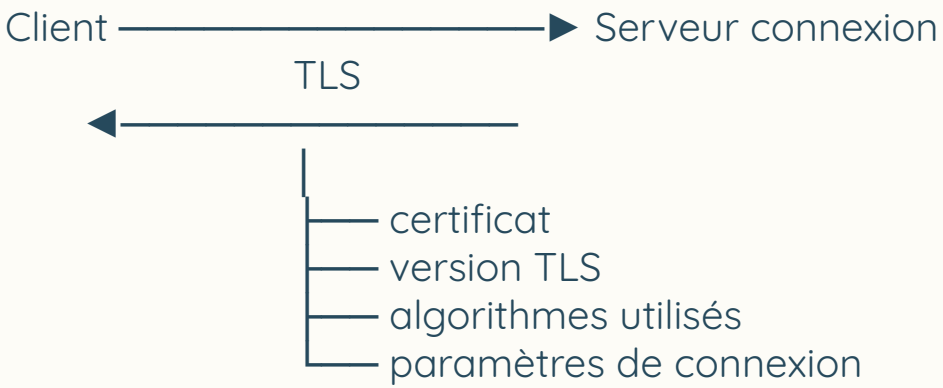
```
$ openssl genrsa -out private.key 2048
```
génère une clé privée RSA

Certificats numériques (document électronique qui permet de prouver l'identité d'une entité) et d'associer cette identité à une clé cryptographique

```
$ openssl x509 -in certificate.crt -text -noout
```
inspecter un certificat (propriétaire, autorité, dates de validité, clé publique, noms de domaines associés)

Tester une connexion TLS (connexion réseau chiffrée et authentifiée)

```
$ openssl s_client -connect example.com:443
```

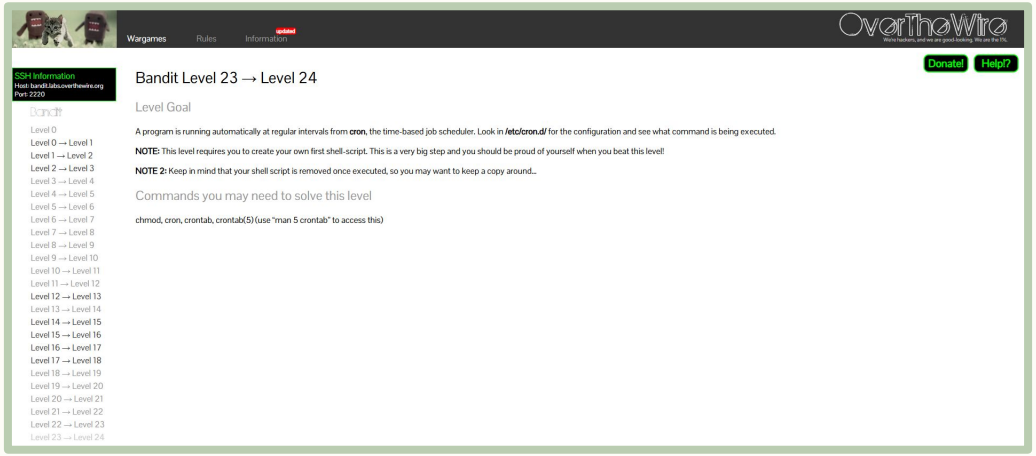


Relativement équivalent à nc pour une connexion réseau mais avec le protocole TLS

Serious Game, OverTheWire

Rendez-vous sur https://overthewire.org/wargames/bandit/bandit0.html

Suivre les instructions et accéder jusqu'au niveau "**Bandit Level 23 → Level 24**"



Variables d'environnement

Une variable d'environnement est une variable accessible par tous les processus fils du shell courant

export sert à rendre une variable d'environnement visible par les programmes lancés depuis le shell courant

`$ export -p` liste tous les noms qui ont été exportés dans le shell actuel

`$LANG` détermine la langue que les logiciels utilisent pour communiquer avec l'utilisateur

`$PATH` le système cherche les commandes tapées dans les dossiers spécifiés par la variable $PATH, dans l'ordre où ils sont indiqués

`$ export PATH="$PATH:/opt/mon-programme/bin"` ajoute /opt/mon-programme/bin au PATH pour les programmes lancés depuis ce shell

`$HOME` emplacement du répertoire personnel de l'utilisateur actuellement connecté

`$ export TEST="Hello"` affiche la valeur "Hello" jusqu'à ce que la session se termine avec la variable $TEST

La persistance des exports est effective lorsqu'ils sont indiqués dans le fichier ~/.bashrc ou ~/.profile

`$ unset NOM` retire la variable de l'environnement

En vrac

`$ alias curdir='pwd | tr "/" "\n" | tail -1'`	**alias** réalise des substitutions abrégées de commandes répétitives et/ou longues à taper dans la console
`$ ln -s ~/rep1/rep2/tp1.txt ../tp1.txt`	Créer un lien symbolique à partir du chemin absolu d'un autre fichier où toute opération sur ce fichier "symbolique" (lecture, écriture, etc.) s'effectue sur le fichier référencé
`$ sleep 10`	Réalise une pause pendant x secondes
`$ shutdown -h now`	Éteint la machine
`$ reboot`	Redémarre la machine
`$ curl https://api.example.com/users`	Tester/interroger des API
`$ wget https://example.com/fichier.zip`	Permet de télécharger des fichiers depuis un serveur HTTP, HTTPS ou FTP